# Leaderboards

*sorted sets, score bands, approximate ranks*

Watch the 4-min video

**In one line:** keep scores in a **sorted set** (Redis ZSET: skip list + hash) so updates, rank lookups and top-N reads are all O(log n); beyond one node, **shard by score band** and add the sizes of higher bands, or estimate **approximate ranks** from a score histogram, keeping exact ranks for the top; time-windowed boards are separate, expiring keys.

## 1 Your rank among 100 million

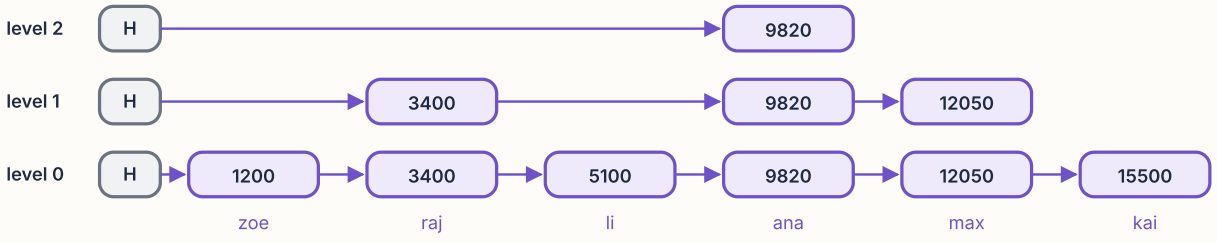
A mobile game has 100 million players. After every match a score changes and the player wants to see "you are #400,012" immediately. In SQL that is `SELECT COUNT(*) FROM scores WHERE score > 9820`: even with an index, the database walks every entry above you, so a mid-table player's rank costs tens of millions of index entries, for every request, while scores keep changing. `RANK() OVER (ORDER BY score DESC)` sorts the whole table. Rank needs a structure that knows *how many* items are before a position without visiting them.

## 2 Redis sorted sets

```
ZADD lb:all 9820 ana              # add or update score       O(log n)
ZINCRBY lb:all 150 ana            # +150 after a match        O(log n)
ZREVRANK lb:all ana               # 0-based rank, high first  O(log n)
ZREVRANGE lb:all 0 9 WITHSCORES   # top 10                    O(log n + 10)
ZREVRANGE lb:all 1402112 1402122  # 5 above + 5 below me      O(log n + 11)
ZCOUNT lb:all 9820 +inf           # players at or above 9820  O(log n)
```

**Figure 1.** The core commands. Ranks are 0-based; ZREVRANK / ZREVRANGE order from highest score (in Redis 6.2+ also `ZRANGE … REV`).

A sorted set maps members to scores and keeps them ordered. Small sets use a compact list; larger ones use a **skip list** (ordered by score, then member) plus a **hash table** (member → score), so both "what is ana's score?" (O(1)) and "what is at rank k?" (O(log n)) are fast.




search starts at the top level and drops down: ~log n hops. Each forward link also stores its span, so the rank of a member = the sum of spans walked to reach it (that's how ZRANK is O(log n), not O(n)).

**Figure 2.** A skip list: sorted linked lists in levels; higher levels skip further. Spans on each link give the rank for free.

**Ties.** Equal scores are ordered by member name, which is rarely fair. Encode a tie-breaker into the score: a ZSET score is a double, which represents integers exactly up to $2^{53}$. With points up to 1 million (20 bits)

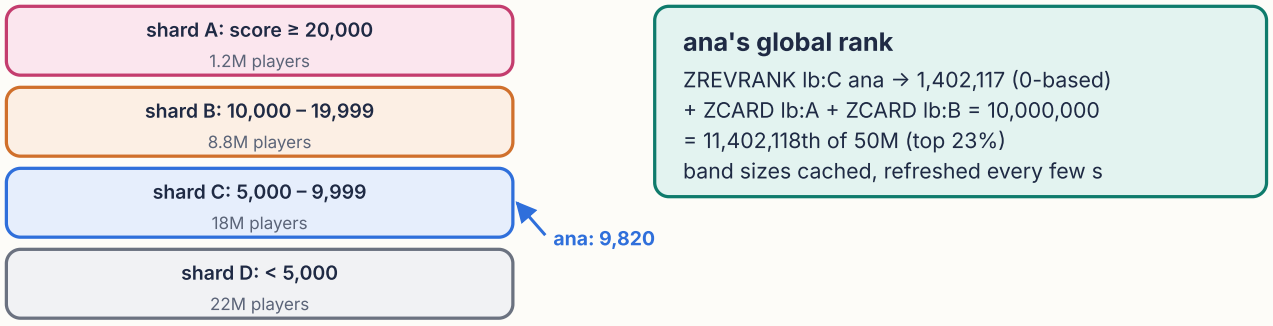
and a 32-bit "seconds until 2100" value: `score = points × 2³² + (T_end − t)`, so among equal points the earlier achiever ranks higher, within 52 bits.

**Around me and friends.** `ZREVRANGE` with rank ± 5 gives neighbours. For a friends board, fetch friends' scores with `ZMSCORE` (or keep a small per-user set) and sort a few hundred values in the app.

## 3 Beyond one node: shard by score band

One sorted set comfortably holds tens of millions of members on one node (memory is very roughly on the order of 100 bytes per member, so 50M members is several GB), but every write for every player hits one single-threaded node. Two ways to split:

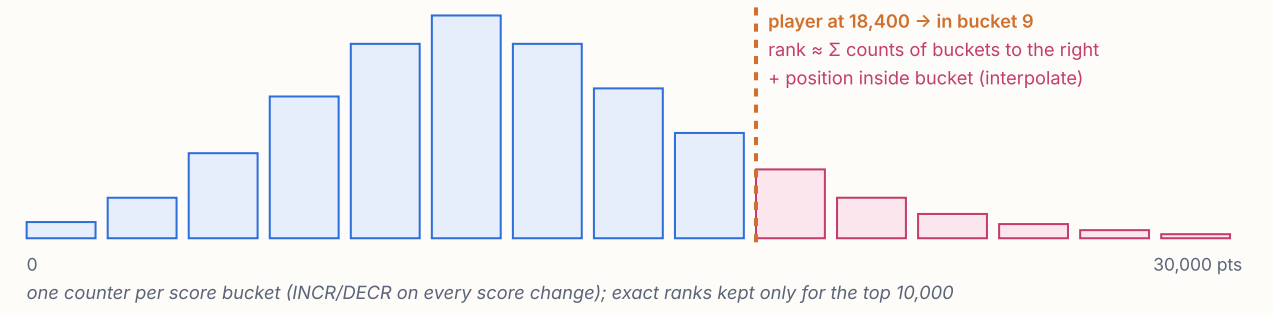
- **By player ID:** writes spread evenly, but a global rank needs every shard to count players above the score: scatter-gather on every lookup. Fine for per-region or per-shard boards.
- **By score range:** each shard holds a band. A player's global rank is their rank within their band plus the sizes of all higher bands (cached `ZCARD`s). A score that crosses a band boundary is a `ZREM` on one shard and a `ZADD` on another.




*bands are chosen from the score distribution so shards hold similar load; low scores are dense, top bands sparse*

**Figure 3.** Score-band sharding. One local rank lookup plus a few cached counts gives an exact global rank.

## 4 Approximate ranks




*one counter per score bucket (INCR/DECR on every score change); exact ranks kept only for the top 10,000*

**Figure 4.** A score histogram. A few thousand counters estimate any player's rank in O(buckets) with no per-player index.

At #11,402,118 nobody notices whether the number is exact; "top 23%" is often more meaningful. Keep a **histogram**: a counter per score bucket, incremented and decremented as scores move. A rank is the sum of counts above the player's bucket, plus an interpolated position inside it. Keep an exact sorted set only for the **top 10,000**, where players do compare positions, and show everyone else a rounded rank or percentile.

| Approach | Rank lookup | Exact? | Use when |
|---|---|---|---|
| SQL COUNT / RANK() | O(rank) or full sort | yes | small boards, offline reports |
| One Redis ZSET | O(log n) | yes | up to tens of millions |
| ZSETs by score band | O(log n) + cached counts | yes | very large boards, exact ranks needed |
| Histogram + top-N ZSET | O(buckets) | top only | huge boards where "top x%" is enough |

## 5 Daily, weekly and all-time boards

Each window is its own key: `lb:daily:2026-10-31`, `lb:weekly:2026-W44`, `lb:all`. Every score update writes to all of them (pipelined); window keys get an `EXPIRE` a few days after the window closes, so old boards clean themselves up, and a reset is just a new key name. A weekly board can also be built from daily keys with `ZUNIONSTORE lb:weekly 7 lb:daily:… AGGREGATE SUM` (or MAX, depending on the game's rules).

Redis is a fast **view**, not the source of truth: write scores durably to a database (after server-side validation against cheating), and be able to rebuild every sorted set from it.

**WORKED EXAMPLE: A GLOBAL BOARD FOR 50 MILLION PLAYERS**

A match ends; the game server validates the result and writes ana's new total, 9,820, to the scores table. A consumer then pipelines `ZADD` to her band shard (C: 5,000–9,999) and to `lb:daily:2026-10-31` and `lb:weekly:2026-W44`. The top 100 come from shard A with `ZREVRANGE 0 99`, cached for one second because millions of clients request it. Ana's rank is her rank in shard C, 1,402,117, plus the cached sizes of A and B (10,000,000): **#11,402,118**, shown as "top 23%" with her five neighbours from shard C.

**TRADE-OFFS**

- Exact ranks for everyone cost memory and hot-node writes; approximate ranks are nearly free.
- Score-band shards give exact global ranks but need rebalancing as the score distribution drifts.
- ID-based shards balance writes perfectly but make global rank a scatter-gather.
- Caching the top 100 for a second removes load but lets it lag by up to that second.

**COMMON MISTAKES**

- Computing rank with COUNT(*) on every request.
- Treating Redis as the only copy of scores.
- Ignoring ties, so equal scores rank alphabetically by player ID.
- Packing score and timestamp into more than 53 bits, silently losing precision in the double.

## 6 Key terms

**Sorted set (ZSET)**: members ordered by score; skip list + hash

**ZREVRANK**: 0-based rank from the highest score

**Score histogram**: counts per score bucket for approximate ranks

**Skip list**: layered linked lists giving O(log n) search and rank

**Score band**: shard holding a range of scores

**Composite score**: points and tie-breaker packed into one number

## 7 Interview questions

**Q1. How do you get a user's rank among 100M players?** BASIC
Redis sorted sets give ZREVRANK in O(log n); shard by score range and add the sizes of higher bands, or estimate approximate ranks from a score histogram, keeping exact ranks for the top.

**Q2. How do you support daily and weekly boards?** BASIC
Separate sorted sets per time window, written on every update, with keys that expire after the window ends.

**Q3. Two players have the same score. Who ranks higher, and how do you implement it?** SCENARIO
Usually whoever reached it first: pack points and an inverted timestamp into one score that fits in 53 bits.

**Q4. The leaderboard Redis node is lost. What happens?** SCENARIO
Scores are durable in the database, so you fail over to a replica or rebuild the sorted sets from the database; ranks are briefly stale, not lost.

**Q5. Why is SQL's COUNT(*) WHERE score > x slow even with an index?** TRICKY
A B-tree doesn't store subtree counts, so it must walk every entry above x; a skip list with spans (or an order-statistic tree) gets the count in O(log n).

**Q6. Why not shard the leaderboard by player ID?** TRICKY
Writes balance well, but a global rank then needs every shard to count players with higher scores on every lookup.

Next: SD-Q-013 · Counting at Scale backendroadmap.com